SL2DT-04: SumoQL → DQL Translation

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 4 of 11 | **Created:** April 2026 | **Last Updated:** 11/04/2026

Overview

Goal of this step: translate the SumoQL query surface (dashboard panels, scheduled searches, saved searches, monitor conditions) into DQL. Use the `sumoql-to-dql` skill as the canonical mapping reference. Track confidence per query; flag LOW-confidence queries for manual review before they ship.

This is the largest engineering effort in the migration — a typical customer has thousands of queries. Get the automation pipeline right before starting; batch-translate; review in rank order of confidence.

Table of Contents

1. What You'll Produce
2. The Translation Engine — Overview
3. Critical Translation Rules
4. Running Batch Translation
5. Confidence Distribution Analysis
6. Handling LOW-Confidence Queries
7. Validating DQL Against Live Tenant
8. Worked Examples — 5 Common Patterns
9. Step Exit Criteria
10. References

Prerequisites

Requirement	Details
Audience	Core migration engineers + app-team reviewers
Skill reference	`sumoql-to-dql` SKILL.md + mapping-tables.md + examples.md
Inputs	Inventory from SL2DT-02 (dashboards.json, monitors.json, searches.json, fers.json)
Dynatrace access	Platform Token with `storage:buckets:read` + `storage:logs:read` + `storage:events:read` + `storage:metrics:read` for validation
Tools	Python 3 for batch translation scripts; optionally a purpose-built migration tool (not yet published)

1. What You'll Produce

Artifact	Purpose
`translations/all-queries.csv`	Every query — source SumoQL + translated DQL + confidence + notes
`translations/high-confidence.csv`	Auto-approved (confidence ≥80)
`translations/medium-confidence.csv`	Requires sample validation
`translations/low-confidence.csv`	Manual rewrite required
`translation-report.md`	Aggregate metrics + escalations

Commit to migration repo. These gate the downstream monitor and dashboard rebuild work.

2. The Translation Engine — Overview

The `sumoql-to-dql` skill is the engine's specification. It defines:

- **Pipeline mapping** — `| parse` / `| where` / `| count by` → DQL equivalents
- **Function mapping** — `pct` → `percentile`, `count_distinct` → `countDistinctExact`, `case` → nested `if`
- **Metadata mapping** — `_sourceCategory` → a bucket and/or a custom attribute such as `sumo.source_category`, per your SL2DT-03 mapping — never a `dt.*` field (SL2DT-03 §4 explains why `dt.source_entity` cannot carry it)
- **Confidence scoring** — deductions for parse syntax conversion, transpose, outlier, etc.

The engine is consumed either:

1. **Manually** — human reads the skill, translates one query at a time. Use for LOW-confidence + complex queries.
2. **Programmatically** — via a purpose-built migration tool (not yet published) when available. This tool batches the translation using the skill as its data source.

Until the tool exists, manual translation with skill reference is the path. The `translate` subcommand equivalent can also be implemented as a short Python script using the mapping tables — see SL2DT-08 for an automation sketch.

3. Critical Translation Rules

Five rules must be applied to every translation. The repository validator rejects queries that break rules 1–3; Grail itself runs them, often with a wrong or empty result — which is why they need applying by hand.

Rule 1 — Explicit Time Range on Every `fetch` / `timeseries`

Sumo inherits the time range from the dashboard or panel. A DQL query without `from:` does not fail — it silently uses the notebook or dashboard timeframe — so the window stops being part of the query and numbers drift from the Sumo panel. Every `fetch logs`, `fetch spans`, `fetch events`, `fetch bizevents`, or `timeseries` should carry a `from:` value.

Rule 2 — Alias Every Aggregation Referenced Downstream

Sumo gives you `_count` automatically. DQL requires explicit aliasing before sort or filter can reference the aggregation.

Rule 3 — Reduce Arrays Before Filter/Sort on `timeseries` Output

`timeseries` returns arrays. You cannot `filter` or `sort` on the array directly — reduce with `arrayAvg`, `arraySum`, `arrayMax`, etc.

Rule 4 — Parse Uses DPL Matchers, Not Sumo Wildcards

Sumo's `parse "user=* status=*"` uses `*` wildcards. DPL uses named matchers: `LD? 'user=' DATA:user ' status=' DATA:status`. Start with `LD?`, not `LD` — `LD` needs at least one character, so it misses lines that begin with `user=`.

Rule 5 — Keyword Search Is Case-Insensitive in Sumo, Not in DQL

A bare Sumo keyword (`_sourceCategory=prod/api ERROR`) matches `error`, `Error` and `ERROR`: *"Keyword expressions are case-insensitive."* `contains(content, "ERROR")` matches only the uppercase form — on a test tenant on 10/05/2026 it found 12,383 of the 76,251 lines that `contains(content, "error", caseSensitive:false)` found in the same hour. Translate a bare keyword as `contains(content, "<keyword>", caseSensitive:false)`. The Sumo `matches` operator is different: *"The string expression is case sensitive"*, so a `matches(_raw, "*ERROR*")` translates to a case-sensitive `contains`. Where the intent is "error logs" rather than the word, use `status == "ERROR"`, which also covers `SEVERE`, `CRITICAL` and the other error levels.

> **Sources:** Keyword search expressions (Sumo Logic docs), matches operator (Sumo Logic docs).

Example — Rules 1–4 Applied

```
// Translation of Sumo:
//   _sourceCategory=prod/api | parse "status=*" as status
//   | timeslice 1m | count by status, _timeslice
//   | where status &gt;= 500 | sort by _count desc
fetch logs, from:-1h                                                    // Rule 1:
explicit from
| filter sumo.source_category == "prod/api"
| parse content, "LD? 'status=' INT:status"                             // Rule 4:
DPL matcher
| makeTimeseries c = count(), interval:1m, by:{status}                  // Rule 2:
aliased to c
| fieldsAdd c_max = arrayMax(c)                                         // Rule 3:
reduce array
| filter status &gt;= 500
| sort c_max desc
```

4. Running Batch Translation

Pull the query surface from `inventory/` JSON files into a flat CSV for translation.

Extract all SumoQL queries

```
# Dashboards
jq -r '[.panels[]? | {
  asset_id: .id,
  asset_type: "dashboard_panel",
  dashboard_id: .dashboardId,
  query: .query
}]' inventory/dashboards/*.json &gt; queries-dashboards.json

# Monitors — the full per-monitor definitions saved in SL2DT-02 §3.2
jq -s '[.. | objects | select(.contentType? == "Monitor") | {
  asset_id: .id,
  asset_type: "monitor",
  query: .queries[]?.query,
  threshold: .triggers[0]?.threshold
}]' inventory/monitors/*.json &gt; queries-monitors.json

# Searches + scheduled searches
jq -r '...' inventory/searches.json &gt; queries-searches.json

# Merge into one
jq -s 'add' queries-*.json &gt; translations/input.json
```

Run the translation pass

When the `Dynatrace-SumoLogic` migration tool exists:

```
migrate.py translate translations/input.json \
  --output translations/all-queries.csv \
  --confidence-threshold 80 \
  --sourcecategory-map taxonomy-map.md
```

Until then, iterate manually with the skill's mapping tables. For a typical 500–2000 query batch, this is 2–5 days of engineer time for core-team examples; app teams do the rest.

CSV Output Schema

Column	Description
`asset_id`	Sumo asset ID (for round-trip correlation)
`asset_type`	dashboard_panel / monitor / search / fer

Column	Description
`sumo_query`	Original SumoQL
`dql_query`	Translated DQL
`confidence`	HIGH / MEDIUM / LOW
`confidence_score`	0–100
`deductions`	Reasons for score reduction (semicolon-separated)
`warnings`	Translator warnings (e.g., "transpose not supported")
`needs_human_review`	true/false

5. Confidence Distribution Analysis

For a typical Sumo deployment, expect this distribution:

Confidence	Typical share	Action
HIGH (80–100)	70–85%	Auto-use; sample-validate during dual-run
MEDIUM (50–79)	15–25%	100% review before shipping
LOW (<50)	2–10%	Manual rewrite; flag for Dynatrace Intelligence redirect where applicable

Red flags

- **HIGH > 95%** — translator may be over-confident; do deeper sampling in validation
- **LOW > 15%** — engagement cost underestimated; re-plan with stakeholders
- **MEDIUM > 40%** — translator may need tuning; report recurring patterns back to the `sumoql-to-dql` skill maintainer

Breakdown by pattern

Track which translation issues cause most deductions:

Pattern	Impact	Example
Parse syntax conversion	-10	`parse "user=*"` → DPL `DATA:user`
Case → nested if	-15	`case(...)` → `if(..., else:if(..., else:...))`
Transpose required	-20	`transpose row x column y`
`outlier` / `predict` / `logreduce`	-30	Redirect to Dynatrace Intelligence
Nested aggregation	-20	`sum(avg(x))` — rejected
Unknown `_sourceCategory`	-10	Taxonomy map incomplete

This table feeds the translator improvement backlog.

6. Handling LOW-Confidence Queries

LOW-confidence queries fall into three buckets. Handle differently:

6.1 Redirect to Dynatrace Intelligence

Outlier / anomaly detection patterns should become anomaly detectors, not DQL queries.

Sumo:

```
_sourceCategory=prod/api | timeslice 1m | count | outlier _count window=10 threshold=3
```

Do: Configure an anomaly detector on a metric derived from the same log source. See SL2DT-05.

Don't: Try to recreate outlier math in DQL. The baseline is brittle and doesn't adapt to seasonality.

6.2 Manual Rewrite

Patterns that DQL supports but the translator couldn't match (transpose, complex join, custom functions).

Example — transpose:

```
_sourceCategory=prod/api | count by status, host
| transpose row host column status
```

No direct DQL. Use:

```
// Alternative: pivot via summarize + per-status aliases
fetch logs, from:-1h
| filter sumo.source_category == "prod/api"
| parse content, "LD? 'status=' INT:status"
| summarize {
    count_200 = countIf(status == 200),
    count_400 = countIf(status == 400),
    count_500 = countIf(status &gt;= 500)
  }, by:{host.name}
| sort count_500 desc
```

6.3 Flag for Retirement

Some LOW-confidence queries serve no business purpose (old debugging queries, abandoned experiments). Review with the owner — they often say "yeah, cut it."

Goal: minimize the hand-rewrite load. For a 500-query batch, after Dynatrace Intelligence redirects and retirement, actual hand-rewrites should be under 30.

7. Validating DQL Against Live Tenant

Every translated query must execute successfully against a live Dynatrace tenant before being added to a notebook, monitor, or dashboard.

Validation tools

```
dtctl verify query --context &lt;ctx&gt; -f -   — syntax and identifier check
(read its WARNING notifications)
mcp__dynatrace__execute-dql              — runs the query; confirms results
and returns notifications
```

Validation via API

```
curl -X POST "$DT_TENANT/platform/storage/query/v1/query:execute" \
  -H "Authorization: Bearer $DT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"query": "fetch logs, from:-5m | limit 10"}'
```

Sample Size

For a 500-query batch:

- HIGH confidence: validate 10% random sample
- MEDIUM confidence: validate 100%
- LOW confidence: validate 100% after rewrite

Failure patterns

Failure	Fix
`Unknown field`	Field name wasn't renamed correctly (check the `sumo.source_category` mapping from SL2DT-03)
`Invalid syntax`	Translator bug — report to skill maintainer, fix by hand
`Query timeout`	Query scans too much data — add `from:` narrower range or `samplingRatio:`
`Empty results`	Data isn't yet flowing for that scope (SL2DT-03 parity issue)
`Empty results` from a bucket typo	Grail does **not** error on an unknown bucket name — `bucket: {"no_such_bucket"}` returns zero rows with no notification. Confirm the bucket exists with `fetch dt.system.buckets | filter name == "…"` (SL2DT-03 §3) before trusting a zero

8. Worked Examples — 5 Common Patterns

Each example shows the source Sumo, target DQL, and confidence. These are representative; the skill's `references/examples.md` has 15 more.

Example 1 — Top errors by host (HIGH 90%)

Sumo:

```
_sourceCategory=prod/api ERROR
| count by _sourceHost
| sort by _count desc
| limit 10
```

DQL:

```
// Example 1 — Top errors by host (HIGH 90%)
fetch logs, from:-1h
| filter sumo.source_category == "prod/api"
| filter contains(content, "error", caseSensitive:false)   // Sumo keywords
are case-insensitive (Rule 5)
| summarize c = count(), by:{host.name}
| sort c desc
| limit 10
```

Example 2 — Latency percentiles by endpoint (HIGH 95%)

Sumo:

```
_sourceCategory=prod/api | parse regex "latency=(?&lt;lat&gt;\d+)"
| pct(lat, 50), pct(lat, 95), pct(lat, 99) by path
```

DQL:

```
// Example 2 — Latency percentiles by endpoint (HIGH 95%)
fetch logs, from:-1h
| filter sumo.source_category == "prod/api"
| parse content, "LD? 'latency=' INT:latency"
| summarize {
    p50 = percentile(latency, 50),
    p95 = percentile(latency, 95),
    p99 = percentile(latency, 99)
  }, by:{http.path}
```

Example 3 — Error rate timeseries (MEDIUM 80%)

Sumo:

```
_sourceCategory=prod/api | timeslice 1m
| if(matches(_raw, "*ERROR*"), 1, 0) as is_error
| sum(is_error) as errors, count as total by _timeslice
| errors / total * 100 as error_pct
```

DQL:

```
// Example 3 — Error rate timeseries (MEDIUM 80%)
// Sumo's matches() is case-sensitive, so a case-sensitive contains() is the
faithful translation here.
fetch logs, from:-1h
| filter sumo.source_category == "prod/api"
| fieldsAdd is_error = if(contains(content, "ERROR"), 1, else:0)
| makeTimeseries {errors = sum(is_error), total = count()}, interval:1m
| fieldsAdd error_pct = 100.0 * errors[] / total[]                // one
value per minute, like _timeslice
```

Example 4 — Lookup enrichment (MEDIUM 75%)

Sumo:

```
_sourceCategory=prod | count by _sourceHost
| lookup environment, owner from path://lookups/host_metadata on
_sourceHost=hostname
```

DQL (requires entity attributes or a Grail lookup table):

```
// Example 4 — Lookup enrichment (MEDIUM 75%)
//
// Corrected 08/12/2026: `dt.tags.&lt;name&gt;` is not a field. Entity tags
are a single `tags` field
// holding an ARRAY of "key:value" strings, so a tag is matched with an
iterative expression —
// iAny(startsWith(tags[], "team:")) — not addressed as a nested field. The
old cell failed with
// FIELD_DOES_NOT_EXIST. Pass the whole `tags` array through the lookup and
read it downstream.
// Join on the host entity ID. `host.name` is not the host entity's name, so a
// host.name → entity.name join matches no host and leaves `tags` empty.
fetch logs, from:-1h
| filter startsWith(sumo.source_category, "prod/")
| summarize c = count(), by:{dt.entity.host}
| lookup [fetch dt.entity.host, from:-7d
         | fields id, entity.name, tags],
    sourceField:dt.entity.host, lookupField:id,
    fields:{entity.name, tags}
```

Example 5 — Outlier (LOW — Dynatrace Intelligence redirect)

Sumo:

```
_sourceCategory=prod/api | timeslice 1m | count | outlier _count window=10
threshold=3
```

Action: configure anomaly detector. DQL baseline is a fallback only if Dynatrace Intelligence can't be used (rare):

```
// LOW CONFIDENCE: prefer an anomaly detector.
// Fallback baseline calc (for reference, not production):
//
// Corrected 08/12/2026: there is no `arrayStdDev` function
(UNKNOWN_FUNCTION). Standard deviation is
// an AGGREGATION, `stdDev()`, so bin the counts and aggregate rather than
building an array first.
fetch logs, from:-1h
| filter sumo.source_category == "prod/api"
| summarize c = count(), by:{minute = bin(timestamp, 1m)}
| summarize mean = avg(c), sd = stdDev(c)
| fieldsAdd upper = mean + 3 * sd
```

9. Step Exit Criteria

G4 — Translation Complete

- [ ] Every query from `inventory/` in `translations/all-queries.csv`
- [ ] HIGH-confidence queries: ≥75% of total
- [ ] MEDIUM-confidence queries: 100% reviewed
- [ ] LOW-confidence queries: classified (Dynatrace Intelligence / rewrite / retire)
- [ ] 10% random sample of HIGH queries validated against live tenant
- [ ] 100% of MEDIUM queries validated against live tenant
- [ ] Translation report delivered to stakeholders

Next step: SL2DT-05 — Monitor/Alert Conversion (apply translations + anomaly detection redirects to Monitor-class assets).

---

10. References

Dynatrace query and pattern languages

- DQL Reference (DT docs)
- DPL Pattern Reference (DT docs)
- Grail (DT docs)

Sumo Logic source-language reference

- Sumo Logic search operators (Sumo Logic docs)
- Sumo Logic parse operators (Sumo Logic docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official Dynatrace documentation and Sumo Logic documentation.